

Storage Wars

Spanner leaves Google's data centres, and Perplexity walks away from DynamoDB

Issue #4

Thu, 08 Oct 2026

~41 min read

Level 3/5

Today's two stories

1. **Google's Spanner now runs outside Google.** Spanner Omni, a version of Spanner you download and run on your own servers, on AWS or on a laptop, became generally available on 30 September. To get there, Google had to replace the two pieces of in-house infrastructure Spanner was famous for needing: the atomic-clock time service called TrueTime and the Colossus file system. We explain what those pieces did, what replaced them, and what you take on when you run it yourself.
2. **Perplexity swapped DynamoDB for a key-value store it built in two months.** CobbleDB, written in Rust on top of RocksDB, now serves the web pages behind Perplexity's search answers. Perplexity reports batch reads about five times faster and storage costs at least 20% lower. We look at why DynamoDB's pricing and opacity hurt this particular workload, how CobbleDB is put together, and what it gave up to get those numbers.
3. **On the radar:** 6 more things that happened, listed at the end.

STORY 1 Spanner Omni: Google's database, without Google's hardware

SPANNER OMNI 2026.R4-LTS · GA 30 SEP 2026 · GOOGLE CLOUD · ~17 MIN READ

For more than a decade, Spanner has come with an unusual footnote. It's the database that made global, strongly consistent SQL look easy, but it did so with help nobody else could buy: GPS receivers and atomic clocks in Google's data centres, and a file system that only exists inside Google. "Just run Spanner yourself" was never an option, because the database leaned on the building it lived in. On 30 September Google shipped a version that doesn't. It runs on your servers, on AWS, or in a Docker container on your laptop, and the clocks it relies on are now plain software.

WHAT HAPPENED

On **30 September 2026**, Google announced that **Spanner Omni**, "the deploy-anywhere version of Spanner, is now generally available, ready to power your most demanding production workloads in your on-premises data center or on other clouds."¹ The release notes call the GA build **2026.r4-lts**. It's a long-term support (LTS) release *≈ a version the vendor commits to patching for an extended period, so you don't have to upgrade every few months* that "receives security fixes for up to one year", and it includes managed Spanner's changes up to the 18 September 2026 release.² InfoQ covered the launch on 7 October.³

Omni was first shown in **preview on 22 April 2026** at Google Cloud Next.^{2,4} Google says it has been downloaded more than 2 million times since then.¹ The road to GA ran through several beta builds, and it wasn't always smooth. Release 2026.r2.1-beta in August didn't support in-place upgrades, so users had to move their data to a new deployment, and 2026.r3-beta in September is a mandatory stepping stone: "Users must upgrade to this release before upgrading to future releases."²

The GA release adds the features you'd expect a production database to have from day one:^{1,2}

- **Security:** TLS encryption, authentication and authorization, and audit logging. Username-and-password login over TLS needs recent client libraries: Java 6.119.0+, Go v1.94.0+ or Python 3.72.0+.
- **Backup and restore**, which Google pitches as protection against "fat-finger" deletes and data corruption.
- **Worker nodes**, which are stateless *≈ holding no data of their own, so they can be added, removed or replaced freely* compute nodes that take background and resource-heavy work off the main servers.
- **Approximate nearest-neighbour vector search** for datasets of up to 1 billion vectors, running on those worker nodes in the Commercial edition.

There are two editions. The **Developer edition** is free for non-production use with a 90-day license, but a single-server deployment of 4 vCPUs or fewer doesn't expire and includes backup and restore. The **Commercial edition** is an annual subscription priced per vCPU *≈ virtual CPU: one processor core as the operating system or hypervisor sees it*, and Google doesn't publish the price.^{1,5}

Google named one GA customer. Attio, a London-based CRM company, runs a containerised Spanner Omni alongside managed Spanner. Its CTO, Alexander Christie, says Omni delivers "Spanner capabilities and performance across all our environments".¹ At the April preview, Mercado Libre said Omni "enables true cross-cloud resilience".⁴

THE BACKGROUND YOU NEED

What Spanner is, and why it's special

Spanner is Google's distributed SQL *≈ a relational database that spreads its data across many machines and keeps full SQL and ACID transactions, instead of making you shard it by hand* database. Work on it started in 2007 as Google looked for an alternative to MySQL. Google described it in a research paper in 2012 and opened it to Cloud customers as a beta on 14 February 2017.⁶

Three ideas do most of the work.

First, Spanner divides your data into pieces and spreads them across servers, splitting and rebalancing them as they grow. This is automatic sharding *≈ the database deciding on its own how to divide data into pieces (shards) and which server holds each piece*.⁴

Second, each piece is copied to several servers, and the copies agree on every change with Paxos *≈ a consensus protocol: a group of servers agrees on each change, and it counts as committed once a majority (a quorum) has accepted it*. One copy acts as the Paxos leader for its piece.⁸ A write is safe once a quorum *≈ a majority of the replicas in a group, for example 2 of 3 or 3 of 5* of copies has it, so losing a minority of servers loses nothing. Google calls this synchronous replication *≈ copying a write to other servers before telling the client it succeeded*.⁴

Third, and this is the part that made Spanner famous, it offers external consistency *≈ a guarantee that if transaction T1 finishes before T2 starts, T2 is ordered after T1 everywhere, as if all transactions ran one at a time in real-time order*. Google's docs put it simply: "the system behaves as if all transactions run sequentially".⁷ Database people also call it strict serializability *≈ the strongest standard isolation level: transactions look serial, and that serial order respects real time*.

Why is that hard? Because a distributed database needs to stamp every transaction with a time, and the clocks on different machines never agree exactly. If server A's clock runs a few milliseconds fast and server B's runs slow, a transaction that really happened later could get an earlier timestamp. Readers would then see a world where effects come before causes.

TrueTime: a clock that admits it's unsure

Google's answer was **TrueTime**. Instead of returning one time, TrueTime returns an interval. Doug Judd, a Spanner engineer, explains: "The method `TT.now()` returns a pair of timestamps `[earliest, latest]` and offers a guarantee that the absolute time ... at which `TT.now()` was invoked falls somewhere in between the returned interval."⁸ The half-width of that interval is usually called epsilon (ϵ) *≈ the clock uncertainty: how far the true time could be from the clock's reading, in either direction*.

In the original design, each data centre had **time masters** with GPS receivers, plus "Armageddon masters" with local atomic clocks in case the satellites became unreachable. A daemon on every machine polled several masters, threw out outliers, and let its advertised uncertainty grow slowly between polls to cover worst-case clock drift. In the 2012 paper, the uncertainty varied "between `1ms` and `7ms`".⁹ By 2023, Google said TrueTime gave Spanner servers "less than 1 millisecond clock uncertainty in the 99th percentile".⁸

Commit wait: buying certainty with a pause

TrueTime alone doesn't fix ordering. Spanner pairs it with a rule called commit wait *≈ a pause at the end of a write: the leader holds back the result until its commit timestamp is guaranteed to be in the past on every clock*. Judd describes the steps: the leader picks `commit_ts = TT.now().latest`, replicates the write through Paxos, and then "waits for `TT.now().earliest > commit_ts`" before releasing locks and answering the client.⁸

Do the maths and you'll see why the uncertainty matters so much. If `now()` is the true time plus or minus ϵ , the leader picks a timestamp ϵ ahead of the real time, and then has to wait until the earliest possible time has passed it. That's roughly 2ϵ after the moment it picked the timestamp. With ϵ at 4 ms, a write would sit for about 8 ms. Once the wait is over, every server anywhere is guaranteed to read a later time, so any transaction that starts afterwards gets a bigger timestamp. Judd's payoff line: "If a transaction T2 calls `Commit()` after T1's call to `Commit()` has returned, then, T2 commit timestamp > T1 commit timestamp."⁸

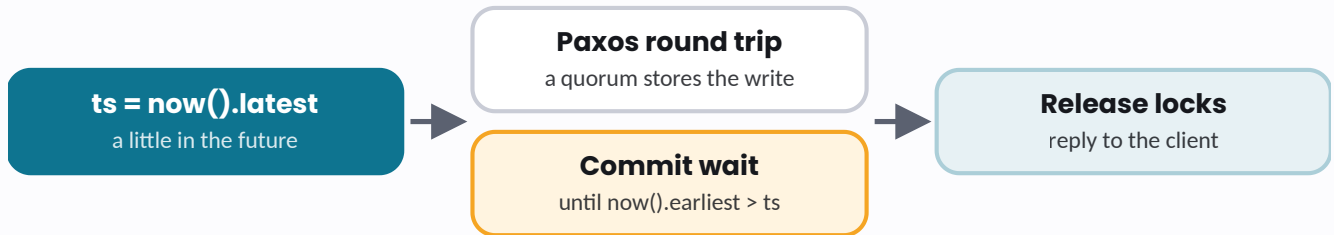
The wait isn't dead time, though, because it runs **at the same time** as the Paxos round trip to the other replicas. In Judd's words, "the vast majority of the time, Spanner's commit wait step involves no waiting".⁸ Keep that sentence in mind. It's the reason Omni is possible at all.

One Spanner write, and where commit wait hides

LEADER PICKS A TIME

IN PARALLEL

DONE



Steps from Google's explanation of commit wait. If the wait finishes before the Paxos round trip does, it adds no latency at all.

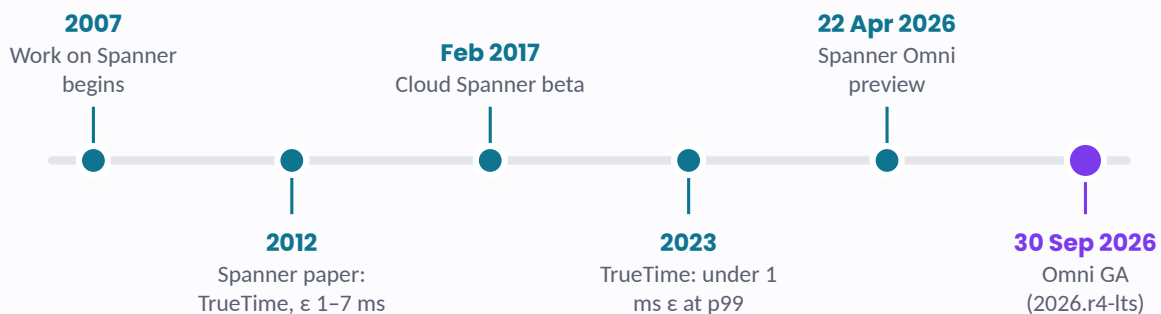
Colossus: the other thing you couldn't take home

The second dependency was storage. Managed Spanner keeps its data on **Colossus** ≈ Google's internal distributed file system, which isn't available outside Google. That means a Spanner server doesn't really "own" its disks: data lives in a shared, replicated storage layer, and compute can move around it.

What other databases did instead

Spanner's rivals had to live without atomic clocks, and they took a different route. CockroachDB, for example, relies on ordinary clock synchronisation plus a configured bound: "The default maximum clock offset value is 500 ms."¹⁰ Instead of waiting out the uncertainty on every write, a reader that runs into a value inside its uncertainty window may have to restart at a later timestamp, and a node whose clock drifts too far shuts itself down "to guarantee database read consistency".¹⁰ That's a sensible design, but it isn't Spanner's design. Omni's question was whether Spanner's own approach (commit wait) could survive on clocks far worse than GPS and atomic clocks.

From Google-only to deploy-anywhere



From TechCrunch (2017), Sookocheff on the 2012 paper, Google Cloud (2023) and the Omni release notes.

HOW IT WORKS

Google kept the core of Spanner (Paxos, automatic sharding, synchronous replication) and replaced the two pieces that tied it to Google's buildings.⁴ Its stated design goal is short: "If you have compute and a database-grade local file system, you should be able to run Spanner Omni."⁴

Time: TrueTime as a piece of software

Omni keeps the TrueTime **API**, the same [earliest, latest] interval, but builds it from ordinary parts. Here's how the Omni documentation describes it:¹¹

- One database server is chosen as the **primary time server**, "the authoritative single source of truth for the entire Spanner Omni deployment". It hands out time "from its local, high-precision clock". There's no separate time infrastructure to install, because the time server is built into the Spanner Omni binary.
- A background **time client** on every host keeps asking the primary for the time and publishes the result to the local Spanner processes.
- Between those queries, each host counts time with a hardware counter. "On Linux x86 architectures, this counter is the **Time Stamp Counter (TSC)** *≈ a register in x86 CPUs that counts clock cycles; reading it is a very cheap way to measure elapsed time*."
- If the primary stops responding, "the cluster dynamically promotes another database server to assume this role".

The width of the interval, the ϵ , now depends on two things: the network round-trip time to the primary, and how much the local clock can drift between synchronisations. So hosts in the same data centre as the primary see smaller uncertainty than distant ones.¹¹ A Google developer advocate's walkthrough adds that the time server sits in the preferred leader zone or region, with several time servers per region that can take over from each other. Because writes usually go to leaders in that zone, uncertainty stays low exactly where it matters.¹²

The docs don't publish typical ϵ values for Omni. What they do publish is the rule that makes the whole idea work: uncertainty "doesn't increase transaction commit latency as long as the uncertainty is smaller than the Paxos round-trip latency."¹¹ That's the same overlap we saw in the diagram. Google's launch post says it outright: because Spanner can "overlap time uncertainty waits with other database work", it can "tolerate weaker uncertainty bounds than TrueTime provides in practice."⁴

In other words, managed Spanner's atomic clocks gave it a lot more precision than it strictly needed. As long as your servers are close enough that a Paxos round trip takes longer than the clock uncertainty, a software clock costs you nothing on writes. If they're far apart, the Paxos round trip is long anyway, which again hides the wait. The case that hurts is the one in between: a time primary that's far away from the writers, or clocks that misbehave.

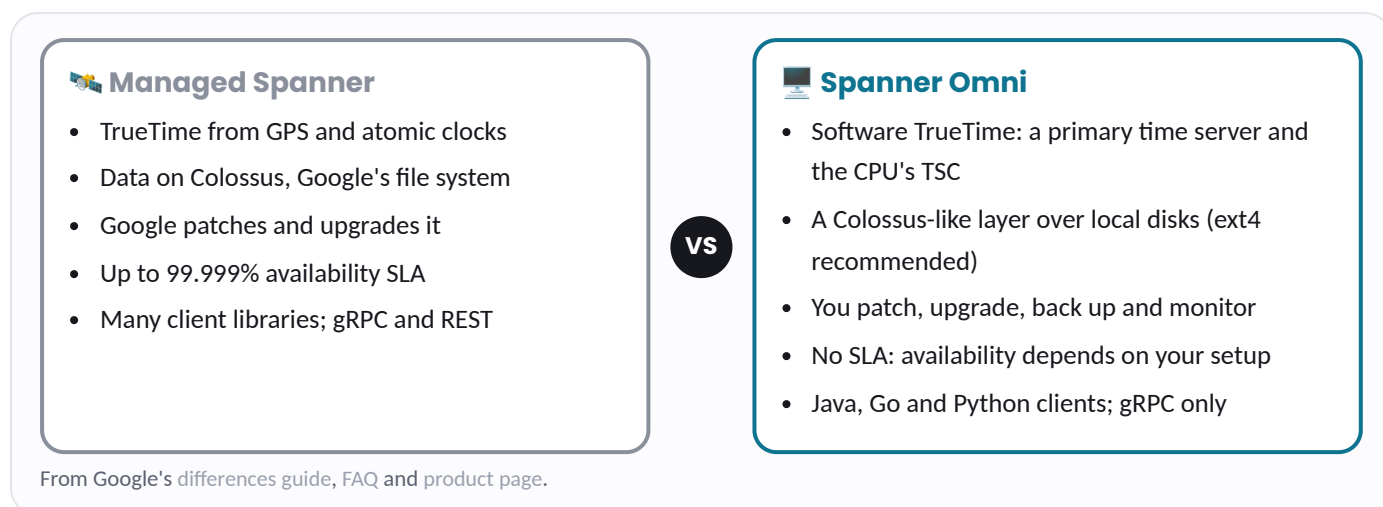
When the clock misbehaves

Because correctness depends on the uncertainty bound being honest, Omni is strict about anything that could break it. The docs list two failure modes:¹¹

1. **TrueTime unavailability**, which "can mean Spanner Omni is also unavailable", either briefly or persistently.
2. **Clock rate error violations**, which can come from **live migration** *≈ moving a running virtual machine to a different physical host without stopping it, which can briefly freeze or skew its clock*, VM suspensions, or a TSC running outside its expected bounds.

As a result, live migration of the VMs or containers running Omni "is not supported", with exceptions for specific machine types and AMIs. Omni exports three metrics to watch in Grafana: `true_time_is_available` (alert on any unavailability), `sla_tester_violation_count` (clock or hardware violations), and `true_time_interval_uncertainty` (€ itself, since high values raise commit latency).¹¹

The beta history shows this isn't theoretical. A June patch fixed "a TrueTime issue where time servers entered repeated failover mode, causing the uncertainty window to spike", and the same patch added an init container to the [Helm chart](#) *≈ a packaged set of Kubernetes configuration files that installs an application* for `/dev/vmclock` on AWS.²



Storage: a Colossus-shaped layer over ordinary disks

For storage, Google wrote what it calls a "Colossus-like" abstraction layer. It "writes to attached local file systems and makes them available across the network to other nodes", and it "automatically handles shard splitting and rebalancing to balance storage across all available servers".⁴ The FAQ says it works on local block devices or [SANs](#) *≈ storage area networks: shared disk arrays that servers reach over a dedicated network* formatted with standard file systems, "with ext4 recommended".¹³ The overview recommends "dedicated solid-state drive (SSD) storage with an ext4 file system".¹⁴

The effect is that, just like on Colossus, a server can read data that physically sits on another server's disk. Storage acts as one pool, separate from compute, so Spanner can move shards around without copying them first.¹² Google is careful not to oversell it: "While Spanner Omni's file layer isn't Colossus, it's a sufficient stand-in to allow Spanner Omni to perform comparably to the Spanner managed service for most workloads."⁴ Its only published performance claim comes from internal benchmarks: "millions of queries per second" across petabytes in a single regional deployment.⁴ That's a vendor claim, without a published setup.

What a deployment looks like

Omni arranges servers into a hierarchy of regions, zones and servers, and the docs describe four shapes:¹⁴

- **Single server**, for local development. "Upgrades to this topology cause downtime."
- **Single zone**: at least three servers. "A single-zone failure can cause an outage."

- **Multi-zone (regional):** at least three zones, with three servers per zone recommended.
- **Multi-cluster (multi-regional):** for high availability, three zones across two or more clusters, with three or more servers per zone.

Those numbers aren't arbitrary. With three copies of each shard, a Paxos quorum is two, so one zone can fail and writes keep going. The minimum hardware is modest: 4 GB of RAM per vCPU and at least 20 GB of disk on RHEL 9 or Ubuntu 22, or 4 vCPUs and 16 GB per VM or pod on Google Cloud and AWS. On an Apple Silicon Mac, 4 GB of RAM and 10 GB of disk is enough to try it.¹⁴

WHAT IT MEANS FOR YOU

If you already run on managed Spanner, Omni's main use is the one Google leads with: a hot-cold failover site in another cloud or your own data centre. Google explicitly mentions financial regulators' "stressed exit" requirements, where a bank must show it could leave a cloud provider.⁵ The other early use is the same database in every environment. Attio's setup, Omni in containers plus managed Spanner, is that pattern.¹

If you need data to stay in a place Google doesn't have a region, Omni is the first way to get Spanner there. Google's launch post mentions multi-regional high availability in jurisdictions where Google Cloud has only one data centre.⁴

If you just want to try Spanner locally, the free single-server Developer edition up to 4 vCPUs doesn't expire. A walkthrough by a Google engineer runs it in Docker, with port 15000 for clients and 15026 for the web console.^{12,13}

If you're comparing it with CockroachDB or YugabyteDB on your own hardware, be clear about what you're buying. Omni is a commercial product with a per-vCPU subscription and no published price, not open source. And you take on everything Google normally does for you. The differences guide is blunt:¹⁵

- You install, patch, back up and maintain the hardware and software. Upgrades are manual, with no downtime for scale-out deployments.
- There's **no SLA**. Managed Spanner offers up to 99.999% availability.⁵
- There's **no built-in encryption at rest**, so you configure disk or file-system encryption yourself. You set up TLS 1.3 for clients and mutual TLS (mTLS) *≈ TLS where both sides present certificates, so servers can verify each other between servers.*
- Logins use passwords with the OPAQUE protocol *≈ a password-authentication protocol in which the server never sees the password itself* or client certificates, and authorization uses an internal role system without custom roles.
- Client libraries exist only for **Java, Go and Python**, and only gRPC *≈ Google's remote-procedure-call framework, which runs over HTTP/2 and uses Protocol Buffers* is supported, not REST.
- Point-in-time recovery (PITR) *≈ restoring a database to its state at any chosen moment within a retention window* is more generous than managed Spanner: up to 30 days of version retention, against 7.

InfoQ quoted one practitioner, Carlos Pérez Martín, who argued that the change is "operational rather than architectural". You'll have to work out quorum placement for your own latency budget, own patching and rollback, and carry backup, key management and audit duties. His advice: run "a like-for-like pilot against the managed service", measured on tail latency and operations work rather than feature parity.³

Watch out for

- **Missing features.** Omni doesn't support BigQuery integration, Data Boost, geo-partitioning, tiered storage, incremental backups, built-in CMEK, the Full-Text Search enhanced query mode, high-scale ANN search, and a few more.¹⁵ Check the list before you plan a failover site that has to behave exactly like production.
- **Outdated docs and blog posts.** InfoQ found the overview page still carrying a Preview banner and preview-era limits two days before GA, such as no TLS, no backups, and writes stopping after 90 days.³ The current overview (last updated 1 October) no longer has the banner.¹⁴ Anything written before 30 September may be describing the beta.
- **Clocks are now your job.** No live migration, alerts on `true_time_is_available`, and a time primary placed close to your write leaders. A badly placed or flapping time server shows up as commit latency.¹¹
- **Upgrade path.** Coming from the beta, you must be on 2026.r3-beta before moving to 2026.r4-lts. Deployments from before 2026.r2.1 had no in-place upgrade at all.²
- **Single-server and single-zone setups** take downtime on upgrade or can go down with one zone.¹⁴ Don't mistake a dev setup for an HA one.

WHAT HAPPENS NEXT

Google says it's working on closing the feature gaps but hasn't given names or dates.¹ The LTS model means 2026.r4-lts gets security fixes for up to a year, so expect the next LTS before that runs out. The two things worth watching are independent numbers, such as ϵ and commit latency on ordinary cloud VMs, which nobody outside Google has published yet, and pricing, which is still "contact sales". Until then, the strongest claim is the architectural one: Google has shown that external consistency doesn't need atomic clocks, as long as the clock's uncertainty stays smaller than the time a Paxos round trip already takes.

Words from this story

Long-term support (LTS) release: a version the vendor commits to patching for an extended period.

Stateless: holding no data of its own, so it can be added, removed or replaced freely.

vCPU: one processor core as the operating system or hypervisor sees it.

Distributed SQL: a relational database that spreads data across many machines while keeping full SQL and ACID transactions.

Automatic sharding: the database deciding on its own how to divide data into pieces and where each piece lives.

Paxos: a consensus protocol where a change counts as committed once a majority of servers accepts it.

Quorum: a majority of the replicas in a group, for example 2 of 3.

Synchronous replication: copying a write to other servers before telling the client it succeeded.

External consistency: if T1 finishes before T2 starts, T2 is ordered after T1 everywhere.

Strict serializability: transactions look serial, and that order respects real time.

Epsilon (ε): the clock uncertainty, how far the true time could be from the reading.

Commit wait: the pause that holds back a write's result until its timestamp is certainly in the past.

Colossus: Google's internal distributed file system.

Time Stamp Counter (TSC): an x86 CPU register that counts cycles, used to measure elapsed time cheaply.

Live migration: moving a running VM to another physical host without stopping it.

Helm chart: a packaged set of Kubernetes configuration files that installs an application.

SAN: a storage area network, shared disk arrays reached over a dedicated network.

Mutual TLS (mTLS): TLS where both sides present certificates.

OPAQUE protocol: password authentication in which the server never sees the password.

gRPC: Google's RPC framework over HTTP/2 using Protocol Buffers.

Point-in-time recovery (PITR): restoring a database to any moment within a retention window.

STORY 2 Perplexity's CobbleDB: why it left DynamoDB, and what it built instead

COBBLEDB · PERPLEXITY ENGINEERING, 14 SEP 2026 · INFOQ, 25 SEP 2026 · ~15 MIN READ

Every time Perplexity answers a question, it has to pull up the web pages behind that answer: not just links, but the actual passages and their vector embeddings, about 50 KB a page, a hundred or so pages per request. Do that 200,000 times a second on a database that charges for every byte you read, and the database stops being a line item and becomes a strategy problem. Perplexity's answer was to write its own key-value store. Two engineers and a swarm of AI coding agents built the core in two months, and Perplexity says it reads about five times faster than DynamoDB did.

WHAT HAPPENED

On **14 September 2026**, Perplexity's engineering team published "CobbleDB: Rebuilding AI search storage for lower latency and cost".¹⁶ InfoQ covered it on 25 September.¹⁷ The post describes how Perplexity moved its search serving tier from **Amazon DynamoDB** to **CobbleDB**, a distributed [key-value store](#) ≈ a database that stores values under unique keys and is optimised for fetching a value by its key, without SQL queries or joins it built in Rust.

The headline numbers, from Perplexity's production traffic at roughly 200,000 requests per second, batches of about 10–15 keys, and an average payload of 50 KB:¹⁶

- **Median** batch-read latency: 31.4 ms → **5.60 ms**
- **p90**: 56.7 ms → **9.77 ms**
- **p99**: 123 ms → **24.2 ms**

That's a 5.08× to 5.80× improvement. Perplexity adds that load tests reached about 500,000 requests per second "with no degradation", and estimates CobbleDB is "at least 20%" cheaper than DynamoDB "at every commitment tier".¹⁶

Two things got as much attention as the numbers. First, the build: Perplexity says the core was written by "two human engineers and hundreds of AI agents in two months", about 40,000 lines of Rust.¹⁶ Second, the future: "We plan to open-source CobbleDB soon," though with no date or license.¹⁶

THE BACKGROUND YOU NEED

What DynamoDB is, and how it bills you

DynamoDB is AWS's fully managed key-value and document database. It launched on 18 January 2012, and it descends from Dynamo, the internal Amazon system described in a 2007 research paper.¹⁸ Its appeal has always been that you never think about servers: you create a table, AWS partitions and replicates it, and you pay for what you use.

"What you use" is measured in request units. In on-demand mode, reads are billed in [read request units \(RRUs\)](#) ≈ DynamoDB's billing unit for reads: one strongly consistent read of up to 4 KB, or two eventually consistent ones, and AWS's pricing page spells out the rules:¹⁹

- "RRUs are consumed in 4 KB increments."
- "Strongly consistent reads consume one RRU per 4 KB or part thereof."
- "Eventually consistent reads require one-half RRUs per 4 KB or part thereof."

An **eventually consistent read** *≈ a read that may return slightly stale data, because it can be served by a replica that hasn't applied the latest write yet* is the default for batch reads. DynamoDB's `BatchGetItem` "performs eventually consistent reads on every table in the request" unless you ask otherwise, and a single call can return "up to 16 MB of data, which can contain as many as 100 items".²⁰

Now put Perplexity's records into those rules. An average item is about 50 KB, which is 13 chunks of 4 KB (rounding up), so even an eventually consistent read costs 6.5 RRUs per item. A batch of 12 or 13 keys costs roughly 80 RRUs. At 200,000 batches a second, that's about 16 million RRUs every second. At the \$0.125 per million reads that AWS uses in its on-demand pricing examples, that's around \$2 a second, or roughly \$7,000 an hour, before any discounts.¹⁹ That's our own back-of-envelope figure, not Perplexity's. Perplexity compared costs across commitment tiers, which are cheaper than on-demand.¹⁶ But it shows the shape of the problem: **with per-byte pricing, big values read at high rates are the expensive case**, and Perplexity's workload is exactly that.

Tail latency and fan-out

The second problem was latency, and specifically **tail latency** *≈ the slowest responses, usually measured as p99: the time that 99% of requests beat*. Perplexity's search API request contains 100–120 page keys, which it splits into batches of 10–20 keys sent in parallel.¹⁶ An answer can't be assembled until **every** batch has come back, so the user waits for the slowest one.

Google's Jeff Dean and Luiz André Barroso described this effect in their 2013 paper "The Tail at Scale". If one server is slow 1% of the time and "a user request must collect responses from 100 such servers in parallel, then 63% of user requests will take more than one second".²¹ That's **fan-out** *≈ splitting one request into many parallel sub-requests and waiting for all of them*: rare slowness on any single call becomes common slowness for the whole request.

On DynamoDB, Perplexity couldn't do much about it. The managed service limited "tuning of partition placement, cache, and replica selection".¹⁶ InfoQ's summary puts it this way: DynamoDB hides partition placement, caching, and replica routing, so the team couldn't control tail-latency spikes.¹⁷

Writes that compete with reads

The third problem was writes. Perplexity continuously crawls the web, and any change to how it chunks pages or computes embeddings means rewriting a lot of records. On DynamoDB, those writes went straight into the same hot store that served user queries, so "large reprocessing jobs competed with hot-store throughput limits".¹⁶ A big batch job could make search slower for users.

RocksDB and LSM trees in one page

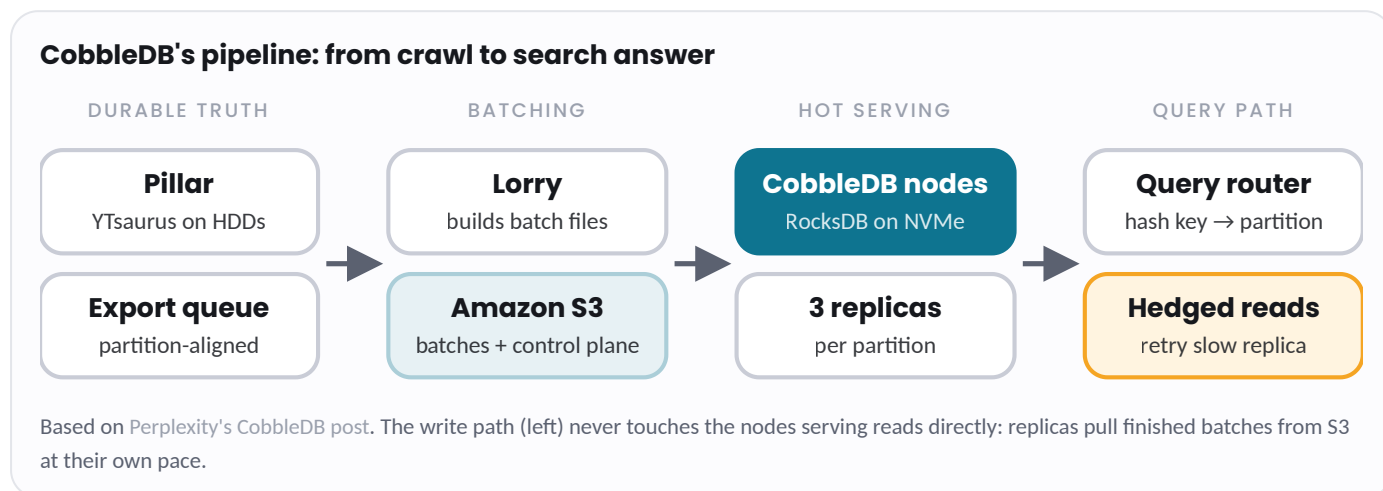
CobbleDB stores its data in **RocksDB**, so it's worth knowing how RocksDB works. CobbleDB uses RocksDB as an embedded engine: it runs inside the CobbleDB process and manages files on the node's local disk.¹⁷ RocksDB is an **LSM tree** *≈ log-structured merge tree: a storage design that buffers writes in memory and writes them to disk as sorted, immutable files, which are later merged in the background* engine.²²

Here's the write path as PingCAP's TiKV documentation describes it.²² Each write first goes to a **write-ahead log (WAL)** *≈ a file where changes are recorded before they're applied, so they can be replayed after a crash*, then into an in-memory sorted structure called a **memtable** *≈ RocksDB's in-memory buffer of recent writes, kept sorted by key*. When the memtable fills up, RocksDB writes it to disk as a **Sorted String Table (SST) file** *≈ an immutable file of key-value pairs sorted by key*. SST files are arranged in levels, "the default is up to 6 levels", and each level is about 10 times bigger than the one above it. Background **compaction** *≈ merging SST files into the next level to drop overwritten and deleted data and keep reads fast* keeps that structure tidy. Hot data blocks sit in an in-memory **block cache** *≈ RocksDB's RAM cache of recently read data blocks from SST files* that evicts the least recently used blocks.

Reading a key means checking the memtable and then possibly several SST files. To skip files that can't contain the key, RocksDB uses **bloom filters** *≈ small probabilistic summaries that can say "this key is definitely not in this file" without reading the file*. It also has a batched read API called **MultiGet**, which looks up many keys in one call. Its implementation pipelines the bloom filter checks for all keys in the batch by prefetching their memory before probing, an idea RocksDB's developers later extended from SST files to the memtable.²³

HOW IT WORKS

CobbleDB is only one of three parts. The design works because Perplexity split "storing the truth" from "serving reads fast", and gave each job its own system.¹⁶



Pillar: the durable source of truth

Pillar keeps the full, durable state of every document and decides what gets published to the hot store. It runs on **YTsauros**, Yandex's big-data storage and processing platform, which Yandex open-sourced under the Apache 2.0 license in March 2023 after developing it since 2010.^{16,24} Pillar stores page metadata, chunks and

embeddings in separate table families and tracks versions. Policy-defined groups of pages, which Perplexity calls **subsets**, decide which updates are exported to CobbleDB.¹⁶

Pillar uses atomic YTsaurus transactions that cover the durable state, the intent to export, and the input it consumed. If a transaction aborts, "no change is made at any layer".¹⁶ That's where the strong consistency lives in this design, and it's deliberately kept off the read path. Pillar and YTsaurus use cheap **HDD** *≈ hard disk drive: spinning magnetic disks, slow for random reads but very cheap per terabyte* storage, while CobbleDB uses fast **NVMe** *≈ a protocol for solid-state drives attached over PCIe; much faster for random reads than older SSD interfaces* drives, so only the documents actually needed for search results are exported to the expensive tier.¹⁶

Lorry: turning a stream of changes into batch files

Lorry is a stateless service that reads export records from a persistent queue that's aligned with CobbleDB's partitions. It builds one batch file per partition, writes it to Amazon S3 under a unique identifier, and registers it with CobbleDB's **control plane** *≈ the part of a system that manages coordination and metadata (what exists, where it lives), as opposed to the data plane that carries the actual data*. That control plane is backed by a PostgreSQL metadata service.¹⁶

This is the fix for "writes compete with reads". Reprocessing a billion pages now produces a pile of files in **object storage** *≈ storage like Amazon S3 that holds whole files (objects) behind an HTTP API; cheap and durable, but too slow for per-request serving*, not a flood of write requests against the serving nodes.

CobbleDB itself: partitions, replicas and RocksDB

The serving tier is a cluster of data nodes. Keys are hashed URLs, and values are pre-chunked passages with per-chunk embeddings.¹⁶ Data is divided into **partitions** *≈ independent slices of the keyspace; here, chosen by hashing the key*, each node holds a subset of them, and every partition has **three replicas on three different nodes**.¹⁶ Perplexity doesn't say how many partitions there are.

Each replica independently asks the control plane for the next S3 batch for its partition and applies updates "in chronological order". A slow or recovering node simply catches up at its own pace, without holding anyone else back.¹⁶ Perplexity doesn't say exactly how a batch file gets into RocksDB.

Inside a node, data lives in RocksDB. Cached data is served from memory and uncached reads go to local NVMe, which lets the team tune how much RAM to buy for each terabyte of disk. Reads use RocksDB's MultiGet "to fetch multiple keys at once".¹⁶ InfoQ's article shows a short Rust function, `multi_get_pages`, wrapping RocksDB's batched multi-get.¹⁷

Inside one CobbleDB read



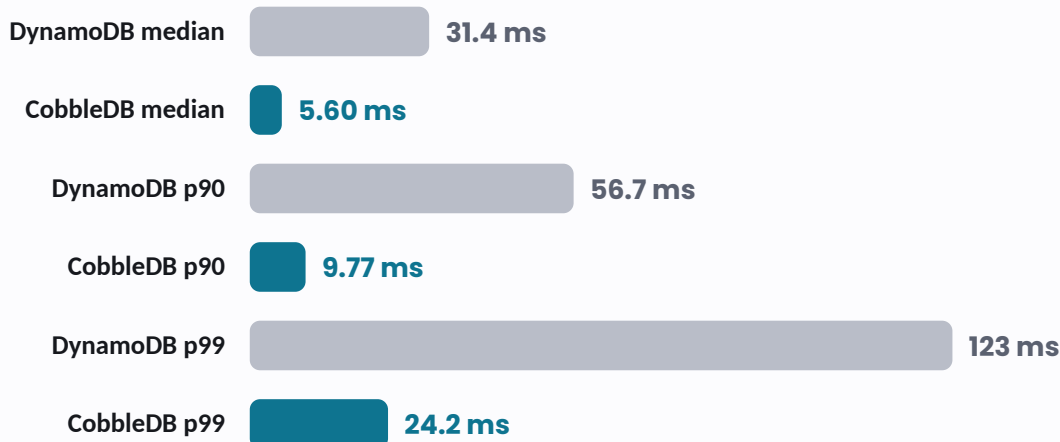
Structure from Perplexity; RocksDB internals from PingCAP's RocksDB overview.

The router and hedged reads

A stateless **query router** hashes each key to its partition and sends the batches to nodes in parallel. It prefers a node in the same **availability zone (AZ)** *≈ an isolated data centre (or group of them) within a cloud region; traffic within one AZ avoids the extra network hops between zones* when it can.¹⁶

And this is where the tail-latency problem gets attacked directly. "If a node is slow, the system can query another partition replica on a different node."¹⁶ That's a **hedged request** *≈ sending a backup copy of a slow request to another replica and using whichever answer comes back first*, straight out of "The Tail at Scale". Dean and Barroso's version is to "defer sending a secondary request until the first request has been outstanding for more than the 95th-percentile", which "limits the additional load to approximately 5% while substantially shortening the tail latency".²¹ Perplexity doesn't publish its hedging threshold. But with three replicas per partition and a fan-out of up to a dozen batches per request, hedging is exactly the tool you'd reach for. On DynamoDB, replica selection was one of the things Perplexity said it couldn't tune.¹⁶

Batch-read latency, DynamoDB vs CobbleDB (ms, lower is better)



Perplexity's own production numbers (source): batches of ~10–15 keys, ~50 KB average payload, ~200k rps. Perplexity calls it an observational before-and-after comparison: the two systems served live traffic at different times.

What it gave up: consistency, on purpose

CobbleDB is fast partly because of what it doesn't do. It "does not provide transactions or synchronized replicas", and Perplexity is explicit about the trade: "It's acceptable for there to be a short gap between when a document is written and when it's available for reads."¹⁶ One replica may be a batch ahead of another, so two reads of the same page a moment apart could, in principle, return different versions. For a search index of web pages that are themselves changing all the time, that's fine. For a bank balance, it wouldn't be.

This is the most important lesson in the design. Perplexity didn't make a faster general-purpose database than DynamoDB. It built a narrow one, with read-only serving nodes, asynchronous batch ingestion, no transactions and no per-write consensus, for a workload that needs exactly that. And it moved the parts that do need guarantees into Pillar, where they're off the hot path.

The AI-agent angle

Perplexity's post spends a lot of space on how CobbleDB was built. An internal swarm of persistent coding agents kept project goals, risks, repository history and earlier decisions across sessions. They watched the project channel, linked open items to pull requests and owners, reviewed code and infrastructure changes, flagged blockers "such as unsafe restore assumptions and stale build references", prepared fixes, tests, monitoring and docs, and tracked CI and rollouts, including backup and restore exercises.¹⁶ Humans "set the architecture, reviewed consequential changes, and authorized production operations".¹⁶ So the "two engineers and hundreds of agents" line is accurate, but the agents didn't design the system or run production on their own.

WHAT IT MEANS FOR YOU

Read the numbers for what they are. They're Perplexity's, from one workload. Perplexity itself calls the latency comparison observational: the systems "served live traffic at different times, not the same requests under controlled conditions".¹⁶ The 500k rps figure comes from load tests, and "at least 20% cheaper" is an internal estimate based on storage size and capacity units, not a bill.¹⁶ They're plausible, and the reasons are well explained, but they're not a benchmark you can apply to your own system.

The useful part is the diagnosis. You can check your own DynamoDB usage against Perplexity's three pain points:

1. **Large items read at high rates.** Per-4 KB billing makes 50 KB values thirteen times as expensive to read as 4 KB ones. If your items are big, look at what you're storing: can you compress values, split hot fields from cold ones, or keep large blobs in S3 and store only pointers?
2. **Wide fan-out with a strict tail budget.** If one user request needs dozens of keys from many partitions, your p99 is driven by the slowest partition, and DynamoDB doesn't let you hedge across replicas.
3. **Batch rewrites sharing capacity with live reads.** If reprocessing jobs slow your users down, separating the write path from the serving path (Perplexity's Pillar → S3 → CobbleDB split) helps whatever database you use.

Building your own is a real cost. Perplexity now owns "node lifecycle management, backup verification, and partition rebalancing", in InfoQ's words, which a managed service would otherwise handle.¹⁷ It had the scale (200k rps) that made the trade worth it. Most teams don't. Grab's recent move of an anti-fraud counter service to Aerospike, with roughly 50% lower p99 read latency and 45–50% lower cost per node, is a reminder that a different off-the-shelf engine plus a redesigned data model is often the cheaper path.²⁵

Watch out for

- **"5× faster" is about this workload.** Batches of 10–15 keys with ~50 KB values, served from three replicas per partition with hedging. It isn't a general verdict on DynamoDB.
- **Eventual consistency is the price.** CobbleDB has no transactions and replicas can lag each other by a batch. Copying the design for data that needs read-your-writes would be a mistake.¹⁶
- **Not open source yet.** "Soon", with no date or license.¹⁶ Until the code is out, you can learn from the design but can't run it.
- **Inflated savings figures are circulating.** Some write-ups quote dollar savings that don't appear in Perplexity's post. The only cost figure Perplexity gives is "at least 20%".¹⁶

WHAT HAPPENS NEXT

The obvious next milestone is the open-source release. When the code is public, the details the blog post leaves out (partition count, how batches are loaded into RocksDB, the hedging threshold) will be visible, and other people will be able to test the latency claims on their own hardware. Until then, CobbleDB is best read as a well-documented case study in a familiar trade-off: managed databases are great until one workload gets big enough, and odd enough, that paying for flexibility you don't use becomes the biggest line on the bill.

Words from this story

Key-value store: a database that stores values under unique keys and is optimised for lookups by key.

Read request unit (RRU): DynamoDB's on-demand billing unit for reads, consumed per 4 KB.

Eventually consistent read: a read that may return slightly stale data from a replica.

Tail latency: the slowest responses, usually measured as p99.

Fan-out: splitting one request into many parallel sub-requests and waiting for all of them.

LSM tree: a storage design that buffers writes in memory and flushes them as sorted, immutable files that are merged later.

Write-ahead log (WAL): a file where changes are recorded first, so they can be replayed after a crash.

Memtable: RocksDB's in-memory, sorted buffer of recent writes.

Sorted String Table (SST) file: an immutable file of key-value pairs sorted by key.

Compaction: background merging of SST files to drop old data and keep reads fast.

Block cache: RocksDB's RAM cache of recently read data blocks.

Bloom filter: a small probabilistic summary that can rule out a key without reading a file.

HDD: hard disk drive, slow for random reads but cheap per terabyte.

NVMe: a fast protocol for PCIe-attached solid-state drives.

Control plane: the coordination and metadata side of a system, as opposed to the data plane.

Object storage: storage like S3 that holds whole files behind an HTTP API.

Partition: an independent slice of the keyspace, here chosen by hashing the key.

Availability zone (AZ): an isolated data centre (or group of them) within a cloud region.

Hedged request: sending a backup copy of a slow request to another replica and using the first answer.

On the radar

Also happened, in one line each. Some will get a full deep dive in a future issue.

- **PostgreSQL 19 RC still pending:** as of 8 October there's no release candidate on postgresql.org, and the roadmap still says the major release "is planned for October 2026".^{26,27}
- **pgx-bm25 1.0 (7 Oct):** a PostgreSQL-licensed extension that adds BM25-ranked full-text search as a native index access method for Postgres 17 and 18, with WAL logging and replication handled by core.²⁸
- **pg_ivm 1.16 (5 Oct):** the incremental materialized view extension adds PostgreSQL 19 support and fixes a segfault when a view is maintained and dropped in the same transaction.²⁹
- **Grab's Counter Service moves to Aerospike:** InfoQ (7 Oct) covers Grab's migration of an anti-fraud counter service, which Grab says cut p99 read latency by about 50% and on-disk data from about 3 TB to 1 TB.^{25,30}
- **Databricks metric view window measures GA (6 Oct):** rolling, cumulative and semi-additive aggregations are now generally available in metric views.³¹
- **Snowflake lineage widens (6 Oct):** data lineage now covers storage integrations, external tables, model monitors and Cortex Search services.³²

1. **Spanner Omni, now GA: A distributed, multi-model database that you can deploy anywhere**, Jagan R. Athreya and Wenzhe Cao, Google Cloud, 30 Sep 2026.

2. **Spanner Omni release notes**, Google Cloud documentation, viewed 8 Oct 2026.

3. **Spanner Omni Reaches GA, Replacing Google's Atomic Clocks and File System with Software**, InfoQ, 7 Oct 2026.

4. **Introducing Spanner Omni**, Wenzhe Cao and Chris Taylor, Google Cloud, 22 Apr 2026.

5. [Spanner Omni](#), Google Cloud product page, viewed 8 Oct 2026.
6. [Google launches Cloud Spanner, its new globally distributed relational database service](#), Frederic Lardinois, TechCrunch, 14 Feb 2017.
7. [TrueTime and external consistency](#), Spanner documentation.
8. [Strict Serializability and External Consistency in Spanner](#), Doug Judd, Google Cloud, 7 Apr 2023.
9. [TrueTime](#), Kevin Sookocheff (summary of the 2012 Spanner paper).
10. [Clock Management in CockroachDB: Good Timekeeping is Key](#), Alex Entin, Cockroach Labs, 22 Apr 2025.
11. [TrueTime and external consistency \(Spanner Omni\)](#), Spanner Omni documentation, viewed 8 Oct 2026.
12. [Spanner Omni: Everything, everywhere, all of the time.](#), Lucía Subatin, Google Cloud Community on Medium, 14 Aug 2026.
13. [Spanner Omni FAQ](#), Spanner Omni documentation.
14. [Spanner Omni overview](#), Spanner Omni documentation, last updated 1 Oct 2026.
15. [Differences between Spanner and Spanner Omni](#), Spanner Omni documentation.
16. [CobbleDB: Rebuilding AI search storage for lower latency and cost](#), Perplexity Engineering, 14 Sep 2026.
17. [Home Made CobbleDB Replaces DynamoDB at Perplexity to Cut Query Latency 5x and Reduce Cloud Storage](#), Olimpiu Pop, InfoQ, 25 Sep 2026.
18. [Amazon's DynamoDB — 10 years later](#), Amazon Science, 18 Jan 2022.
19. [Amazon DynamoDB pricing for on-demand capacity](#), AWS, viewed 8 Oct 2026.
20. [BatchGetItem](#), Amazon DynamoDB API Reference.
21. [The Tail at Scale – Dean and Barroso 2013](#), Adrian Colyer, The Morning Paper, 15 Jan 2015.
22. [RocksDB Overview](#), PingCAP TiDB documentation.
23. [MultiGet batching in memtable \(#5818\)](#), RocksDB commit (mirror).
24. [Yandex open sources YTsaurus, its flagship storage and processing platform](#), Yandex, 20 Mar 2023.
25. [Migrating Counter Service storage: Design choices and learnings](#), Grab Tech Blog, 3 Jul 2026.
26. [News archive](#), postgresql.org, viewed 8 Oct 2026 (latest core release: 19 Beta 4, 24 Sep).
27. [Roadmap](#), postgresql.org, viewed 8 Oct 2026.
28. [pgx-bm25 1.0: BM25 ranked full-text search as a native PostgreSQL index](#), postgresql.org, 7 Oct 2026.
29. [pg_ivm 1.16 released](#), postgresql.org, 5 Oct 2026.
30. [Grab Redesigns Counter Service Storage for 50% Lower P99 Latency](#), Leela Kumili, InfoQ, 7 Oct 2026.
31. [Databricks release notes: October 2026](#), Databricks.
32. [Snowflake server release notes and feature updates](#), Snowflake documentation.